

Sesión 6: Modelado Orientado a Objetos para Árboles

Introducción a Estructuras No Lineales, Terminología de Árboles, Relaciones Autorreferenciadas y
Diseño de Diagramas de Clases UML en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

| Agenda del Día (1/2)

- Estructuras Lineales vs Estructuras Jerárquicas
- 🌿 Anatomía y Terminología de un Árbol
- ↕ Conceptos de Raíz, Hojas, Altura y Grado
- 🔗 Modelado Visual con Diagramas de Clases UML



Pensamiento Jerárquico

Rompiendo la secuencia unidimensional para representar jerarquías orgánicas de información en memoria dinámica.

Agenda del Día (2/2)



Clases de C#

Implementación de tipos autorreferenciados utilizando punteros lógicos nativos del lenguaje C#.



Árboles Binarios

Restricción estructural de grado dos: preparación algorítmica para los modelos ABB y AVL.

Buenas Prácticas

Evitar fugas de memoria y asegurar el correcto encapsulamiento orientado a objetos en .NET.

Transición al Paradigma No Lineal

Por qué las listas secuenciales ya no bastan para problemas complejos

El Límite de las Estructuras Lineales

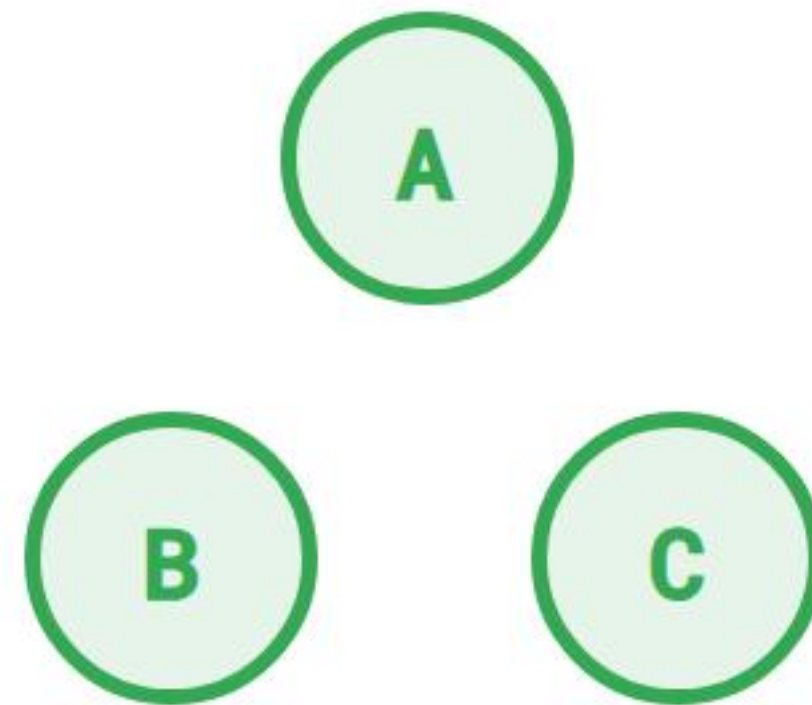
Hasta hoy, todas nuestras estructuras (Listas Simples, Dobles, Pilas, Colas, Matrices) compartían una característica común: ****eran lineales****.

El Defecto Secuencial:

Cada elemento posee un único sucesor directo y un único antecesor directo. Esto imposibilita modelar relaciones de subordinación natural, como organigramas de empresas, sistemas de archivos de discos duros o árboles genealógicos.

¿Qué es un Árbol en Ciencias de la Computación?

Un **Árbol** es una estructura de datos no lineal, dinámica y jerárquica, compuesta por un conjunto de elementos llamados **Nodos**, interconectados por medio de líneas lógicas conocidas como **Aristas o Ramas**.



Un árbol no puede contener ciclos cerrados: **siempre existe un único camino para llegar a cualquier nodo**.

Terminología Esencial de Árboles

El vocabulario técnico formal que define las propiedades jerárquicas

1. El Nodo Raíz (Root)

Es el origen supremo de la estructura. Todo árbol posee **un único nodo raíz** a partir del cual se ramifican los demás elementos de la colección.

- Carece por completo de nodos ancestros o padres.
- Es el punto de entrada exclusivo para cualquier algoritmo de búsqueda o recorrido en el árbol.



2. Relaciones de Parentesco: Padre e Hijo

Las conexiones definen dependencias directas en la jerarquía vertical:

Nodo Padre (Parent)

Es aquel nodo que contiene enlaces apuntando directamente a elementos del nivel inferior inmediato.

Nodo Hijo (Child)

Es el nodo que se encuentra conectado de forma descendente a un elemento del nivel superior.

3. Nodos Hojas y Nodos Internos

Clasificación de los elementos según su capacidad de ramificación terminal:

Clasificación del Nodo	Condición de Estructura	Ejemplo / Representación
Nodo Hoja (Leaf)	No tiene ningún hijo. Sus punteros descendientes apuntan a null.	Extremo final del árbol
Nodo Interno (Internal)	Tiene al menos un hijo. Posee enlaces activos a subárboles.	Nodos de transición intermedia

4. El Grado (Degree)

El **Grado** mide el potencial o capacidad de ramificación ramificada del sistema:

- **Grado de un Nodo:** Es el número total de hijos directos que posee dicho nodo.
- **Grado del Árbol:** Es el grado máximo registrado entre todos los nodos que componen la estructura general.

Si un nodo "A" tiene 3 hijos y todos los demás tienen menos de 3, el grado de ese nodo es 3 y el **grado total** de todo el árbol es 3.

5. Niveles de Profundidad

Los niveles representan la distancia secuencial medida en pasos desde el origen supremo:

```
Nivel 0 → Nodo Raíz  
Nivel 1 → Hijos directos de la Raíz  
Nivel 2 → Nietos de la Raíz (Hijos del Nivel 1)  
Nivel N → Incremento por cada paso jerárquico descendente
```


6. Altura (Height)

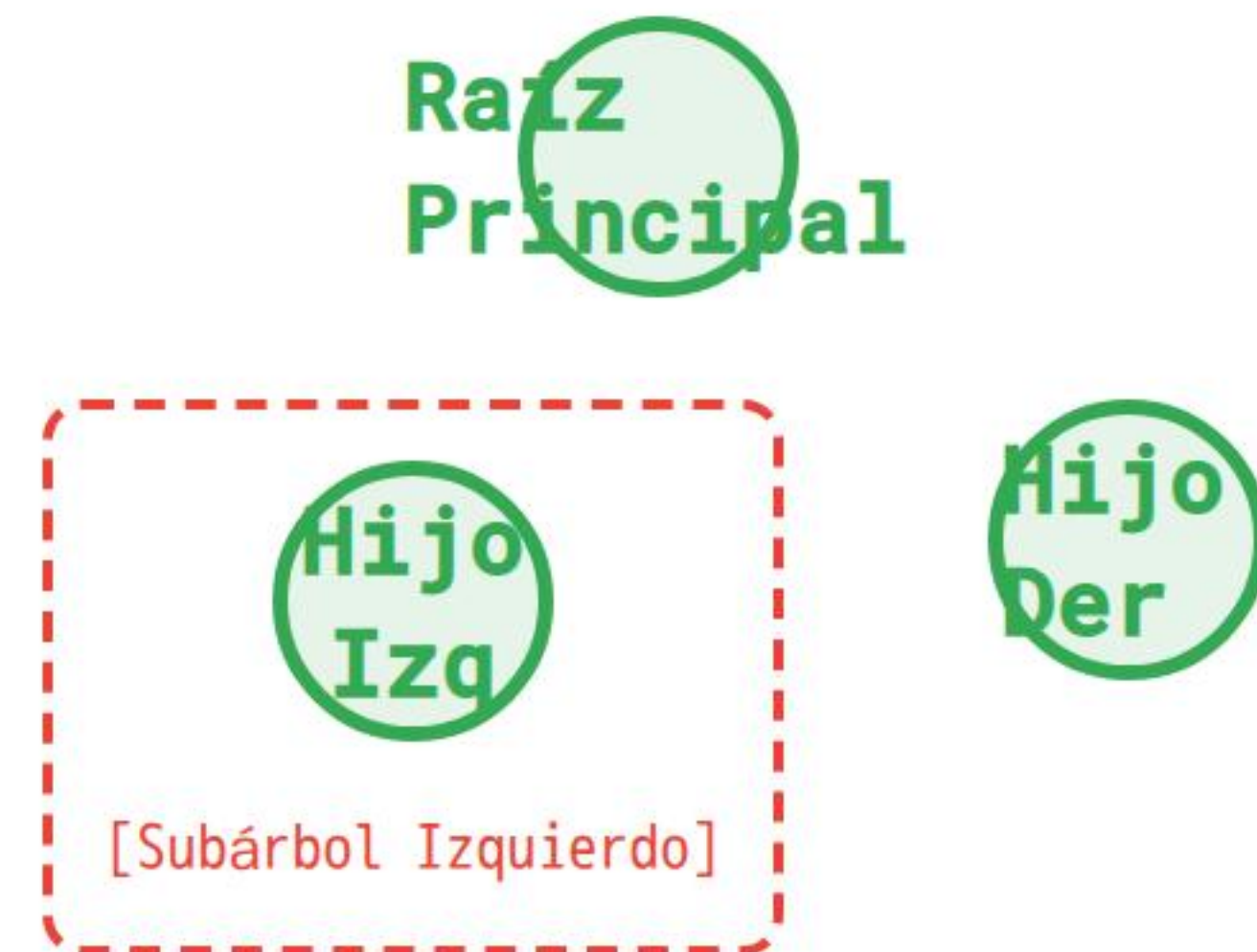
La **Altura** es una de las métricas más críticas para calcular eficiencias asintóticas en las próximas sesiones.

Definición: Es el número máximo de aristas (ramas) que existen en el camino desde la raíz hasta la hoja más profunda del árbol.

Nota académica: Un árbol vacío posee una altura de -1 , mientras que un árbol que solo cuenta con el nodo raíz posee una altura de 0 .

7. El Concepto de Subárbol

Debido a la naturaleza recursiva de los árboles, ****cada nodo hijo puede ser visualizado y tratado como la raíz de su propio subárbol****.



Árboles Binarios

El estándar fundamental de grado dos restringido para el manejo de datos óptimos

¿Qué es un Árbol Binario?

Es un tipo especial de árbol donde cada nodo tiene ****como máximo dos hijos****. Ningún elemento puede violar esta regla bajo ninguna circunstancia.

Subárbol Izquierdo

La rama o puntero que referencia la bifurcación lógica izquierda del nodo padre.

Subárbol Derecho

La rama o puntero que referencia la bifurcación lógica derecha del nodo padre.

Modelado de Estructuras Autorreferenciadas

Comprendiendo los punteros lógicos que se apuntan a sí mismos en la memoria del sistema

¿Qué es una Clase Autorreferenciada?

En la programación orientada a objetos (POO), una clase es **autorreferenciada** cuando contiene campos o propiedades cuyo tipo de dato coinciden exactamente con el de **la clase que se está definiendo**.

Para armar un nodo de un árbol binario en C#, requerimos que el nodo conozca los datos a guardar y, a su vez, tenga dos campos del tipo "Nodo" para apuntar a sus respectivos hijos.

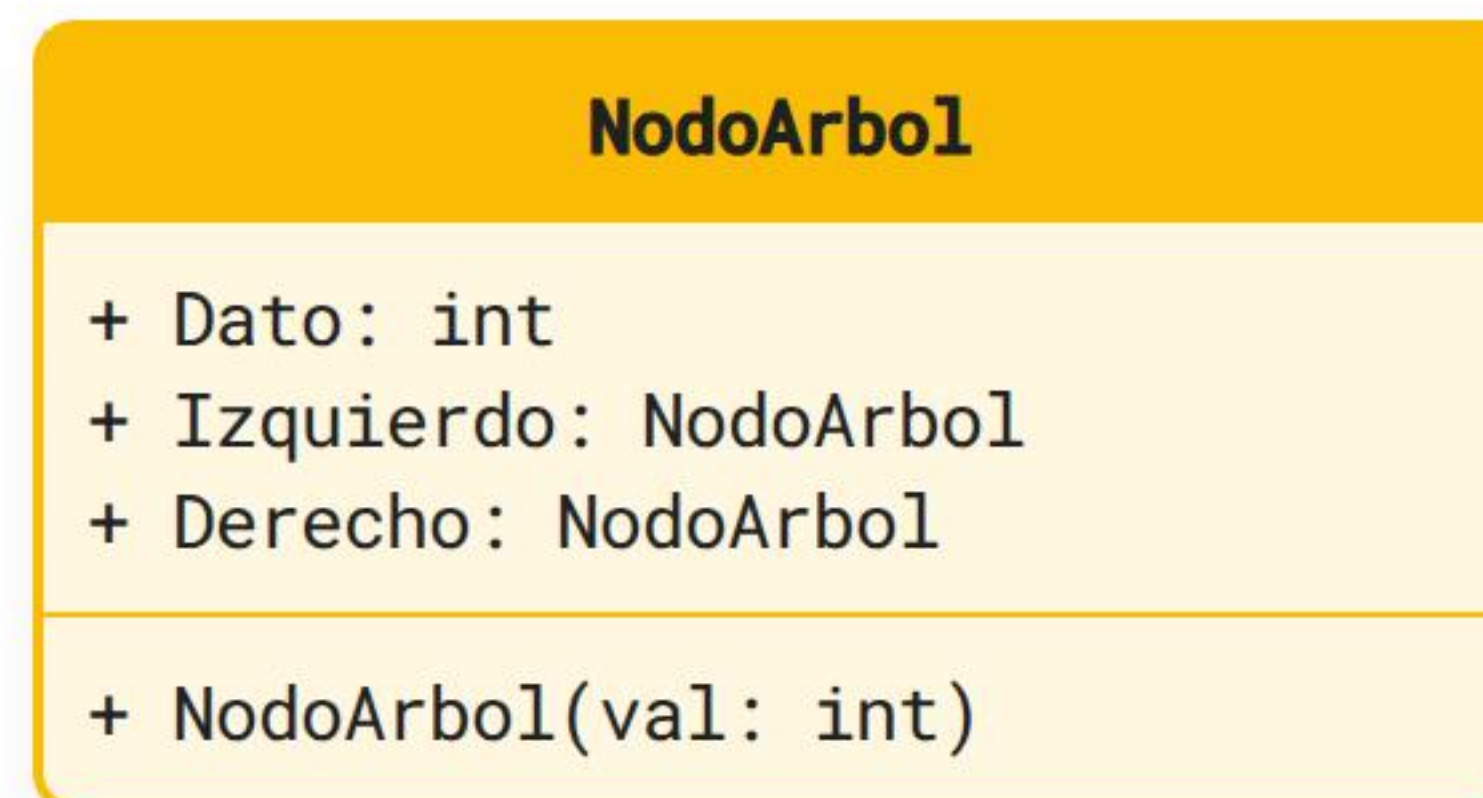


Diseño de Diagramas de Clases UML

Representación visual estándar de las relaciones autorreferenciadas

UML de un Nodo de Árbol Binario

De acuerdo con los estándares de la especificación UML 2.5, la autorreferencia se dibuja con una línea asociativa que nace de la clase y ****retorna hacia sí misma****:



Estructura Completa: Clase Controladora

El árbol completo requiere separar la lógica del nodo de la clase que administra la raíz:

ArbolBinario

- Raiz: NodoArbol

+ Insertar(val: int): void

+ Buscar(val: int): bool

Relación de Asociación:

La clase ArbolBinario tiene una relación de agregación o composición con la clase NodoArbol, manteniendo un puntero exclusivo hacia la **Raíz** suprema.

Traducción a Código Fuente C# .NET

Construcción paso a paso de los bloques estructurados de la sesión 6

Código: Definición del Nodo Autorreferenciado

Implementación limpia y tipada del contenedor elemental de datos en C#:

```
public class NodoArbol {  
    public int Dato { get; set; }  
    // Campos Autorreferenciados nativos  
    public NodoArbol Izquierdo { get; set; }  
    public NodoArbol Derecho { get; set; }  
  
    public NodoArbol(int dato) {  
        Dato = dato;  
        Izquierdo = null; // Inicialización segura  
        Derecho = null;  
    }  
}
```


Análisis Técnico de las Propiedades

Al reservar memoria para un nuevo `NodoArbol` en el Heap:

- El entero `Dato` guarda el valor útil de negocio de forma física.
- Las propiedades `Izquierdo` y `Derecho` nacen con valor `**`null`**`, lo cual define que el nodo, en su estado inicial, se comporta como una `**Hoja terminal**` del árbol.

Código: Clase Controladora Principal

Estructura de la clase encargada de gobernar el árbol y encapsular el acceso directo:

```
public class ArbolBinario {  
    // Atributo privado para proteger el encapsulamiento  
    private NodoArbol raiz;  
  
    public ArbolBinario() {  
        raiz = null; // El árbol inicia vacío  
    }  
  
    // Los métodos de inserción y búsquedas se implementarán en la Sesión 7  
}
```

Buenas Prácticas e Integridad en Memoria

Evitando errores clásicos de punteros en la programación de árboles

Prevención de NullPointerException

Dado que navegaremos a través de referencias, el peligro latente es intentar acceder a las ramas de una hoja terminal:

```
// Código altamente peligroso sin validación previa:  
nodo.Izquierdo.Izquierdo = new NodoArbol(10);
```

⚠ Si `nodo.Izquierdo` es nulo, el sistema colapsará inmediatamente. ****Regla de oro de la Sesión 6:**** Todo acceso direccional debe estar blindado por un bloque condicional que verifique que el puntero no sea `null`.

Valor de la Unidad: Responsabilidad



[cite_start]"No se puede escapar de la responsabilidad del mañana evadiéndola hoy." [cite: 48]

[cite_start]

— **Abraham Lincoln [cite: 48]**

[cite_start]

****Aplicación en Ingeniería:**** Diseñar estructuras siguiendo estándares UML formales antes de codificar demuestra responsabilidad profesional con la calidad de nuestro software[cite: 7].

Conclusiones Generales

Salto Jerárquico

[cite_start]

Los árboles nos liberan de las ataduras de las colecciones secuenciales lineales para abrir camino a clasificaciones de datos más complejas[cite: 43].

Punteros Dobles

[cite_start]

Los árboles binarios limitan su grado a un máximo de dos hijos por elemento (Izquierdo y Derecho), sentando las bases de la optimización informática[cite: 16].

UML Estándar

[cite_start]

Modelar mediante diagramas de clases autorreferenciados permite predecir el flujo lógico del software antes de escribir una sola línea de código[cite: 45].

Próximo Paso: Sesión 7

[cite_start]

Árboles Binarios de Búsqueda (ABB) [cite: 46]

[cite_start]

En la siguiente sesión tomaremos este modelo base y le inyectaremos lógica algorítmica para realizar ****inserciones ordenadas y búsquedas optimizadas**** de llaves[cite: 47]. ¡Asegúrate de tener claros estos conceptos base!

¿Dudas o Preguntas de la Sesión 6?

¡Muchas gracias por su atención!

✉ Soporte: 3021894750101@ingenieria.usac.edu.gt

💬 Foros oficiales en la plataforma UEDI